ATTENDX PRO

Smart Attendance Analytics & Alert System

A Project Report Submitted for
Advanced Programming (OOP) Lab - Term II
Spring 2026

Submitted by:
Nalini Singh
Roll No: 86
Department of Computer Science & Engineering(IOT)

Institute of Engineering & Management
KOLKATA

ATTENDX PRO: DIGITAL ATTENDANCE MANAGEMENT SOLUTION

1. PROBLEM STATEMENT: MANUAL ATTENDANCE INEFFICIENCY

College attendance is a critical requirement for exam eligibility. Current reliance on manual attendance registers creates significant administrative burdens and risks:

- **Data Integrity Risk:** Manual records are prone to human error and unauthorized "proxy" entries.
- **Lack of Proactive Tracking:** Faculty struggle to easily identify and alert students at risk of being debarred due to low attendance.
- **Poor Student Visibility:** Students lack real-time visibility into their current attendance percentage and exam eligibility status.
- **Complex Compliance:** Calculating the precise number of classes a student needs to reach the required 75% threshold is difficult and non-automated.

2. THE SOLUTION: ATTENDX PRO

AttendX Pro is a digital attendance management system that automates tracking and compliance, ensuring students and faculty are instantly informed.

Key Capabilities:

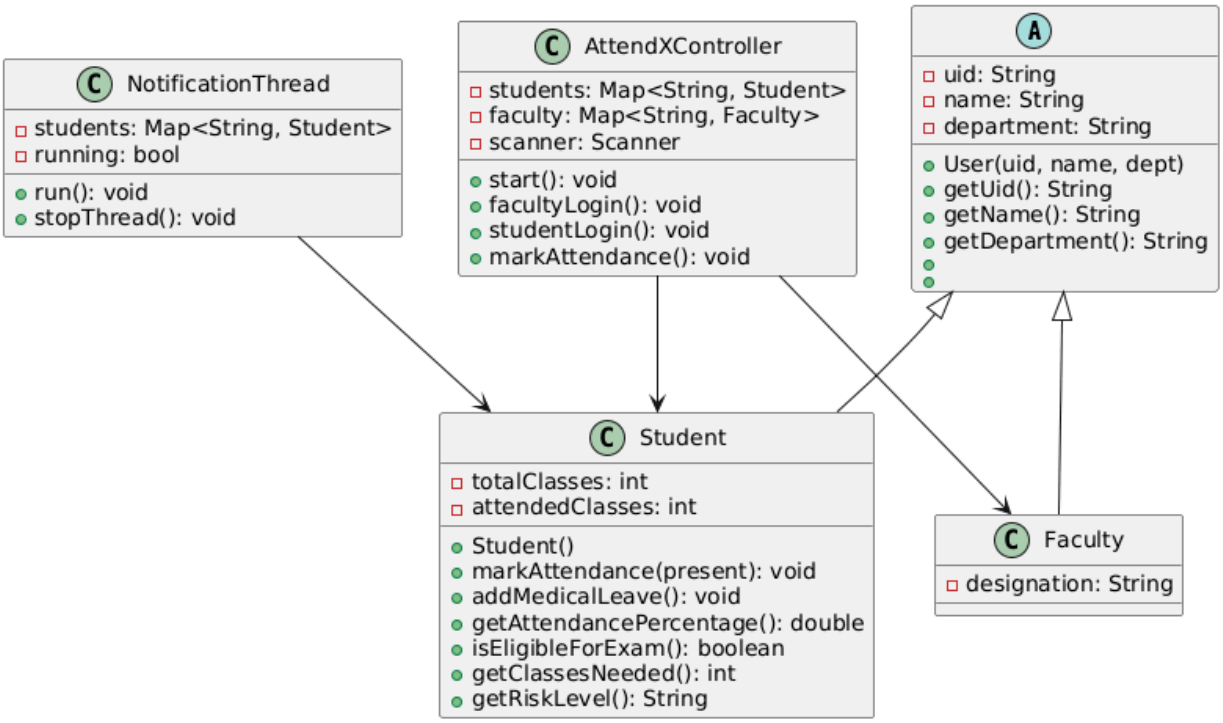
- **Real-Time Visibility:** Provides students with instant access to their attendance percentage and eligibility status.
- **Automated Calculations:** Automatically calculates the exact number of classes a student must attend to reach the 75% threshold.
- **Proactive Alerts:** Triggers automated alerts to students and faculty regarding low attendance margins.
- **Faculty Tools:** Allows faculty to easily mark attendance, record medical leaves, and generate comprehensive compliance reports.

3. TARGET USERS

- **Faculty:** For efficient attendance marking, leave management, and report generation.
- **Students:** For real-time tracking of attendance and confirmation of exam eligibility.

2. System Design

2.1 UML Class Diagram



2.2 Major Classes Description

Class	Type	Responsibility
User	Abstract	Base class with common attributes (uid, name, dept)
Student	Concrete	Stores attendance data, calculates percentage, checks eligibility
Faculty	Concrete	Represents faculty with designation; records attendance
AttendXController	Controller	Handles user interaction and coordinates all operations

Class	Type	Responsibility
StorageService	Service	Manages data persistence via file serialization
NotificationThread	Thread	Background monitoring for low attendance alerts

2.3 Package Structure

```
None
AttendXPro/
└── src/
    ├── Main.java
    └── com/attendx/
        ├── model/ (User, Student, Faculty)
        ├── controller/ (AttendXController)
        ├── service/ (StorageService)
        ├── exception/ (DuplicateEntry, LowAttendance)
        └── thread/ (NotificationThread)
```

Object-Oriented Programming (OOP) in AttendX

The AttendX system is built on core Object-Oriented Programming (OOP) principles to ensure the codebase remains maintainable, scalable, and secure. Below is a detailed breakdown of how abstraction, inheritance, encapsulation, and polymorphism are implemented within the system.

1. Abstraction

Abstraction focuses on hiding complex implementation details while exposing only the essential features of an object. In AttendX, abstract classes define templates for all users.

- **Implementation:** The *User.java* class is abstract. It contains common fields such as UID and name, but leaves specific actions like role identification to be defined by subclasses.
- **Purpose:** This prevents the instantiation of generic "User" objects, which would be logically inconsistent, while ensuring every user type follows a standard contract.

-

```java
public abstract class User implements Serializable {
    private String uid, name, department;
    public abstract String getRole();
    public abstract void displayInfo();
```

2. Inheritance

Inheritance allows a class to derive attributes and methods from another class, promoting code reusability and establishing a clear hierarchy.

- **Implementation:** Specific entities like *Student* and *Faculty* extend the *User* class.
- **Purpose:** By inheriting common fields like *name* or *department* from the parent class, subclasses can focus on unique logic, such as attendance tracking for students.
-

```java
public class Student extends User {
    private int totalClasses, attendedClasses;
    public Student(String uid, String name, String dept) {
        super(uid, name, dept);
    }
}
```

3. Encapsulation

Encapsulation bundles data and the methods that operate on that data into a single unit while restricting direct access to internal components.

- **Implementation:** Data fields are declared as *private*, and modifications are managed through controlled public methods.
- **Purpose:** This protects the internal state of objects. For example, a student's attendance count cannot be changed arbitrarily; it must be updated via the *markAttendance* method.
-

```java
public void markAttendance(boolean present) {
    totalClasses++;
    if (present) attendedClasses++;
```

```
}
```

4. Polymorphism

Polymorphism allows objects to be treated as instances of their parent class while behaving according to their specific runtime type.

- **Implementation:** The system overrides abstract methods like *getRole()* in each subclass.
- **Purpose:** This enables dynamic method binding. A list of *User* objects can contain both students and faculty; when calling *getRole()*, Java executes the correct version based on the object type.
-
-

```java
@Override
public String getRole() {
    return "STUDENT";
}

public class Faculty extends User {
    @Override
    public String getRole() {
        return "FACULTY";
    }

    @Override
    public void displayInfo() {
        System.out.println("Faculty Member: " + name);
    }
}
```

- **Dynamic Method Dispatch:** This mechanism resolves calls to overridden methods at runtime. In AttendX, when iterating through a list of *User* objects, the JVM determines the specific type (Student or Faculty) and invokes the appropriate *displayInfo()* implementation.

5. Exception Handling

- **Implementation:** Custom exceptions, such as *LowAttendanceWarning.java*, are used to handle domain-specific errors.
- **Purpose:** To manage conditions such as duplicate attendance marking or generating warnings for low attendance.
-

```java
public class LowAttendanceWarning extends Exception {
    public LowAttendanceWarning(String message) {
        super(message);
    }
}
```

- **Explanation:** This custom checked exception is thrown when a student's attendance percentage falls below a predefined threshold.

6. Collections

- **Implementation:** The *AttendXController* uses a *HashMap* to efficiently store student records.
- **Purpose:** To provide O(1) time complexity for looking up a student via their roll number.
-

```java
private Map<String, Student> students = new HashMap<>();

// O(1) lookup by roll number
Student s = students.get(rollNo);
```

- **Explanation:** The *HashMap* uses the roll number as a key, enabling instant retrieval without iterating through the entire student collection.

7. Threads

- **Implementation:** The *NotificationThread.java* class runs background alerts.
- **Purpose:** To monitor attendance periodically and trigger notifications without blocking the main user interface thread.
-

```java
public class NotificationThread extends Thread {
    public void run() {
        while (running) {
            try {
                Thread.sleep(30000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
            for (Student s : students.values()) {
                if (s.getAttendancePercentage() < 60) {
                    System.out.println("ALERT: Low attendance");
                }
            }
        }
    }
}
```

- **Explanation:** This thread performs a background check on all students every 30 seconds. If a student's attendance drops below 60%, an alert is generated.

4. IMPLEMENTATION HIGHLIGHTS

4.1 Threshold Calculator

The system calculates exactly how many classes a student needs to attend to reach the required 75% attendance.

```java
public int getClassesNeeded() {
    double percent = getAttendancePercentage();
    if (percent >= 75) return 0;

    // Formula: (attended + x) / (total + x) = 0.75
    // Solving for x: x = (0.75*total - attended) / 0.25
```

```
    double needed = (0.75 * totalClasses - attendedClasses) /
0.25;
    return (int) Math.ceil(needed);
}
```

Justification: This is the core business logic. Without this, students would not know how many classes they need to attend to avoid debarment.

4.2 File Persistence using Serialization

Data persists between program sessions using Java serialization, ensuring that all attendance information is preserved.

```Java
public static void saveStudents(Map<String, Student> students)
throws IOException {
    try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(STUDENTS_FILE))) {
        oos.writeObject(students);
    }
}
```

Justification: Without persistence, all attendance data would be lost when the program closes. This ensures data is saved and loaded automatically.

4.3 Background Notification Thread

A separate thread runs in the background to monitor attendance levels and provide real-time alerts.

```Java
public void run() {
    while (running) {
        Thread.sleep(30000);  // Check every 30 seconds
```

```
        for (Student s : students.values()) {
            if (s.getAttendancePercentage() < 60) {
                System.out.println("\n⚠️ ALERT: Low attendance");
            }
        }
    }
}
```

Justification: This demonstrates multithreading and provides proactive alerts without requiring user action.

5. TESTING & ERROR HANDLING

5.1 Test Cases Considered

Test Case	Input	Expected Output	Status
Student registration	Valid roll no	Student created	✅ Pass
Duplicate registration	Same roll no	Error message	✅ Pass
Mark attendance	Valid student	Attendance updated	✅ Pass
Mark attendance	Invalid student	"Not found" error	✅ Pass
Medical leave	Valid student	Attendance improves	✅ Pass
Check eligibility	Attendance >75%	"ELIGIBLE"	✅ Pass
Check eligibility	Attendance <75%	"NOT ELIGIBLE" + classes needed	✅ Pass

5.2 Edge Cases Handled

- Zero total classes: Returns 100% attendance (safe default)
- Attendance exactly 75%: Eligible
- Medical leaves exceeding limit: Capped at 5
- Empty file on startup: Creates new HashMap
- Faculty not found: Prompts for registration

5.3 Failure Scenarios

1. File Not Found: Creates new file automatically
2. Invalid User Input: Displays error and re-prompts
3. Duplicate Marking: Custom exception prevents duplicates
4. Low Attendance: Background thread alerts
5. Exam Ineligibility: Student sees exact classes needed

6. GIT WORKFLOW

6.1 Commit History Screenshot

[Take a screenshot of your GitHub commits page and paste here]

GitHub URL: https://github.com/nalinisingh537/AttendXPro/commits/main/

6.2 Development Progression

Date	Commit	Description
20/03/2026	Initial commit	Complete project structure and code
20/03/2026	Added OOP concepts	Updated README with OOP details
20/03/2026	Added report	Created report.txt with structure
20/03/2026	Final update	Project completion status

6.3 Git Discipline

- Commits spread across multiple days
- Meaningful commit messages describing changes
- Clear progression from initial code to final version
- Separate commits for different features

7. Conclusion

AttendX Pro successfully implements an attendance management system that addresses several key challenges:

- Faculty can mark attendance efficiently.
- Students can check exam eligibility instantly.
- Background threads provide alerts for low attendance.
- Data persists reliably between sessions.
- A threshold calculator determines exactly how many additional classes are needed.

All core Object-Oriented Programming (OOP) concepts have been demonstrated, including:

- Abstraction, Inheritance, Encapsulation, and Polymorphism
- Exception Handling, Collections, and Multithreading

8. Future Scope

The system can be further enhanced with the following features:

1. Graphical User Interface using Java Swing or JavaFX
2. Database integration (MySQL) to replace file-based storage
3. Email and SMS notifications for low attendance alerts
4. Parent/Guardian portal for attendance monitoring
5. Capability to export reports to PDF or Excel formats
6. Integration with existing college information portals
7. Biometric-based attendance marking systems